Prompt:

I want you to derive LTL constraints from a list of sentences. Each sentence may contain multiple constraints, but they all fit into one of the following five templates:

- Response: A -> F(B)

- Precedence: NOT(B) U A

- Succession: (A -> F(B)) AND (NOT(B) U A)

- Init: START -> A

- End: END -> A

A and B here are symbols for the entities that the constraint is defined on. You may also swap their order. Here are explanations and example sentences for each constraint type:

- Response: When A occurs, B must always occur at some point afterwards. Example sentence: "Once a claim has been approved, it should eventually be paid out.". With "approve claim" as A and "pay out claim" as B, this translates to A -> F(B).

- Precedence: A must always occur before B occurs. Example sentence: "A claim should be created before it can be approved". With "create claim" as A and "approve claim" as B, this translates to NOT(B) U A.

Succession: A combination of response and precedence, so A must eventually be followed by B and B cannot occur until A has occured. Example sentence: "Once a claim has been rejected, it should not be paid out.". With "reject claim" as A and "not pay out claim" as B, this translates to (A -> F(B)) AND (NOT(B) U A).

Init: A needs to occur at the start of a process. Example sentence: “Every claim has to be created in the beginning.” With "create claim" as A, this translates to START -> A.

End: A needs to occur at the end of a process. Example sentence: “After a claim has been paid, it is closed.”. With “pay claim" as A, this translates to END -> A.

I will also give you a list of all possible entities that you can extract from the text. Please stand by.

Response:
Absolutely, I'm standing by. I'm

Prompt:
Here are the entities, these should replace A and B as symbols in the results.

create claim

creation claim

approve claim

reject claim

payout

pay out claim

create case

arrange meeting

enter metadata

submit case

assign case id

agree date

examine patient

check x-ray risk

perform x-ray

perform surgery

apply cast

book ticket

payment ticket

check-in flight

receive invoice

decide invoice legit

receive all necessary approvals

confirm payment

receipt of an invoice

enter data

pay invoice

send request

give confirmation

register client data

provide room service

provide laundry service

leave

search report

bring in a defective computer

each of these activities

error is detected

register claim

Check recommendation

Mark as not ok

Mark as Ok

Receive request for new hardware

Analyze request

Deny request

Approve request

Receive offer

this is accepted

take care of invoice

receive customer request

need further review

submit office supply request

register requirement

reject request

ask to make change

select vendor

approval, rejection, change

submit vacation request

review order

inform customer

wait for x days

inform about delay

deliver package

Receive order

reserve part

ship bicycle to the customer

Receive amount request

Request approval

File expense

Receive expense report

check take place

supervisor approves

supervisor rejects

respond

approval claim

pay claim

case closed manually

manage case

hold meeting

inform

get local id

propose dates

perform x-ray risk

perform reposition

prescribe rehabilitation

remove cast

provide personal information

booking phase completion

money transfer

modify provided data

cancellation

enter all relevant data

close case

give approval

prepare proposal

proposal

bill

charge bill

put in a file together with the claim documentation

analyze the defect

test proper system functionality

execute repair activity

examine claim

send it back

Proceed with claim

inform employee via email

edit request

fill in form

decide if approved

receive hardware by IT department

Send back request to IT

receive request

return to petitioner/employee

go to purchase department

generate purchase order

send notification

return application

generate notification

order missing products

check again

process delivery

update system

assemble bicycle

Assess credit risk

Accept credit

Perform check

stop process

deposit of reimbursement

receives a notification

send email

I will now provide you the sentences in batches of 10.

Response:
Understood. I'm ready to process the batches of

Prompt:
A claim should be created, before it can be approved.

A claim must be created, before it is approved.

When a claim is created, it may be approved.

If a claim is created, it can be approved.

They have to create the claim, before they can approve it.

Only after a claim is created, it is possible to approve the claim.

If a claim is approved, then it must have been created first.

To approve a claim, it should be created first.

A claim can be approved, after it has been created.

Approving the claim can happen at anytime, unless it still needs to be created.

Response:
These sentences correspond to the Precedence template (NOT(B) U A), where B is "approve claim" and A is "create claim".

Here are the LTL constraints:

NOT(approve claim) U create claim

NOT(approve claim) U create claim

NOT(approve claim) U create claim

NOT(approve claim) U create claim

NOT(approve claim) U create claim

NOT(approve claim) U create claim

NOT(approve claim) U create claim

NOT(approve claim) U create claim

NOT(approve claim) U create claim

NOT(approve claim) U create claim

All the sentences imply that a claim must be created before it can be approved. Therefore, the logical structure is the same for all these sentences.

Prompt:
Creation of the claim is required, before it can be approved.

The creation of the claim is a condition for the approval of the claim.

Only after the creation of a claim, the approval of the claim is possible.

When approval of the claim occurs, creation of the claim must have preceded.

If approval of the claim occurs, then creation of the claim must have been carried out first.

The approval of the claim requires that the creation of the claim occurs before it.

The approval of the claim should be preceded by the creation of the claim.

A claim must be created, before it can be approved or rejected.

A claim should be approved before it can be paid out.

Once a claim has been approved, it should eventually be paid out.

Response:
The sentences correspond to the following templates:

1-7. Precedence (NOT(B) U A)

8. Precedence (NOT(B) U A) and Precedence (NOT(C) U A)

Precedence (NOT(B) U A)

Response (A -> F(B))

Here are the corresponding LTL constraints:

NOT(approve claim) U creation claim

NOT(approve claim) U creation claim

NOT(approve claim) U creation claim

NOT(approve claim) U creation claim

NOT(approve claim) U creation claim

NOT(approve claim) U creation claim

NOT(approve claim) U creation claim

NOT(approve claim) U creation claim AND NOT(reject claim) U creation claim

NOT(pay out claim) U approve claim

approve claim -> F(pay out claim)

The entities "creation claim", "approve claim", "reject claim", and "pay out claim" are applied according to the context of the sentences.

Prompt:
Once a claim has been rejected, it should not be paid out.

Payout should end the process and therefore exclude everything.

A case can be closed manually at any time, unless we still need to pay out the claim.

The case can be managed at the union after it has been created.

After a case is created, management can and must arrange a meeting between the union case worker and the other case worker.

After a meeting is arranged it must be held.

The case worker should enter metadata on the case, inform about when he is available for participating in a meeting and then submit the case.

When a case is submitted it may get a local id at the union, but it should also subsequently be assigned a case id in LO.

When a case is submitted, LO should eventually propose dates.

Only after LO has assigned its case id it is possible to manage the case and for LO to propose dates.

Response:
The sentences correspond to the following templates:

Succession ((A -> F(B)) AND (NOT(B) U A))

End (END -> A)

Succession ((A -> F(B)) AND (NOT(B) U A))

Response (A -> F(B))

Response (A -> F(B))

Response (A -> F(B))

Response (A -> F(B))

Response (A -> F(B)) and Response (A -> F(C))

Response (A -> F(B))

Precedence (NOT(B) U A) and Precedence (NOT(C) U A)

Here are the corresponding LTL constraints:

(reject claim -> F(NOT(pay out claim))) AND (NOT(NOT(pay out claim)) U reject claim)

END -> payout

(create case -> F(NOT(case closed manually))) AND (NOT(NOT(case closed manually)) U create case)

create case -> F(manage case)

create case -> F(arrange meeting)

arrange meeting -> F(hold meeting)

enter metadata -> F(submit case)

submit case -> F(get local id) AND submit case -> F(assign case id)

submit case -> F(propose dates)

NOT(manage case) U assign case id AND NOT(propose dates) U assign case id

The entities are "reject claim", "pay out claim", "create

Prompt:

Please only return the LTL constraints, not a list of templates or entities. The next 10 sentences:

Once a date has been agreed upon a meeting should eventually be held.

Every process instance starts by examining a patient.

If an x-ray is taken, then the x-ray risk must be checked before it

Performing a reposition, applying a cast and performing surgery require that x-rays are performed before.

If a surgery is performed, then prescription of rehabilitation occurs eventually after it.

After a cast is applied, eventually the cast is removed and vice versa, before every removal of a cast, a cast must be applied.

The process begins with the booking of the ticket

Personal information of passengers can be provided after the booking of the ticket

The payment of the ticket triggers the completion of the booking phase.

The payment of the ticket is eventually followed by the actual transfer of money.

Response:

agree date -> F(hold meeting)

START -> examine patient

NOT(perform x-ray) U check x-ray risk

NOT(perform reposition) U perform x-ray AND NOT(apply cast) U perform x-ray AND NOT(perform surgery) U perform x-ray

perform surgery -> F(prescribe rehabilitation)

apply cast -> F(remove cast) AND NOT(remove cast) U apply cast

START -> book ticket

NOT(provide personal information) U book ticket

payment ticket -> F(booking phase completion)

payment ticket -> F(money transfer)

Prompt:

As long as the Check-in of the flight does not take place, the customers can still modify the provided data to change

After check-in of the flight, only the cancellation is admitted

The process starts when an invoice is received by the administration department.

If they decide that the invoice appears legit then they enter all relevant data into the system

When all necessary approvals have been received the invoice can be paid.

Once payment is confirmed, the invoice case should be closed.

The receipt of an invoice is required before the administration department can enter data or scan the invoice.

Enter data is required before any approval can be given.

Finally, we have to pay the invoice before we can confirm payment.

If a request is sent, then a proposal is expected to be prepared afterwards.

Response:

NOT(check-in flight) U modify provided data

check-in flight -> F(cancellation)

START -> receive invoice

decide invoice legit -> F(enter all relevant data)

receive all necessary approvals -> F(pay invoice)

confirm payment -> F(close case)

NOT(enter data) U receipt of an invoice AND NOT(scan invoice) U receipt of an invoice

NOT(give approval) U enter data

NOT(confirm payment) U pay invoice

send request -> F(prepare proposal)

Prompt:

A confirmation is supposed to be mandatorily given after the proposal, and vice-versa any proposal is expected to precede a confirmation.

Every process instance has to start by registering client data.

Every provided room service must be billed.

Every provided laundry service must be billed

When the client leaves, the bill must be charged.

The Police Report related to the car accident is searched within the Police Report database and put in a file together with the Claim Documentation.

A customer brings in a defective computer and the manager analyzes the defect and gives back an invoice.

After each of these activities, the proper system functionality is tested.

If an error is detected, another arbitrary repair activity is executed, otherwise the repair is finished.

After a claim is registered, it is examined by a claims officer.

Response:

proposal -> F(give confirmation) AND NOT(give confirmation) U proposal

START -> register client data

provide room service -> F(bill)

provide laundry service -> F(bill)

leave -> F(charge bill)

search report -> F(put in a file together with the claim documentation)

bring in a defective computer -> F(analyze the defect) AND bring in a defective computer -> F(give back an invoice)

each of these activities -> F(test proper system functionality)

error is detected -> F(execute repair activity)

register claim -> F(examine claim)

Prompt:

This recommendation is then checked by a senior claims officer who may mark the claim as OK or Not OK.

If the claim is marked as Not OK, it is sent back to the claims officer and the recommendation is repeated.

If the claim is OK, the claim handling process proceeds.

The process of buying new hardware is started as soon as a request for new hardware is received by the IT department.

The next step is for the IT department to analyse the request and either approve or deny it.

If it is denied, the employee is informed via email about the denial and the process is ended.

However, if the request is approved it is edited and completed.

Once this offer is received, a form is filled in and sent to the management.

The management then analyses the request and decides if it is approved or not.

When this is accepted and the financial resources are found, the hardware is received by the IT department.

Response:

Check recommendation -> F(Mark as Ok) AND Check recommendation -> F(Mark as not ok)

Mark as not ok -> F(send it back)

Mark as Ok -> F(Proceed with claim)

Receive request for new hardware -> F(Analyze request)

Analyze request -> F(Approve request) AND Analyze request -> F(Deny request)

Deny request -> F(inform employee via email)

Approve request -> F(edit request)

Receive offer -> F(fill in form)

fill in form -> F(decide if approved)

this is accepted -> F(receive hardware by IT department)

Prompt:

After which the financial department will take care of the invoice and the processes is ended.

In the scenario where the management does not approve the request, it can send the request back to the IT department for rework, then the IT department has to go back to editing and completing a request.

If the management does not approve the request, it can also decide to deny the request after which the process is ended.

The loan approval process starts by receiving a customer request for a loan amount.

If the customer needs further review or the loan amount is for \$10,000 or more, the request is sent to the approver Web service.

The process of an Office Supply Request starts when any employee of the organization submits an office supply request.

Once the requirement is registered, the request is received by the immediate supervisor of the employee requesting the office supplies.

If the request is rejected, the process will end.

If the request is asked to make a change, then it is returned to the petitioner who can review the comments for the change request.

If the request is approved, it will go to the purchase department that will be in charge of making quotations (using a subprocess) and select a vendor.

Response:

take care of invoice -> F(END)

NOT(decide if approved) -> F(edit request)

NOT(decide if approved) -> F(END)

START -> receive customer request

need further review -> F(submit office supply request)

submit office supply request -> F(register requirement)

register requirement -> F(receive request)

reject request -> F(END)

ask to make change -> F(return to petitioner/employee)

approve request -> F(select vendor)

Prompt:

After a vendor is selected and confirmed, the system will generate and send a purchase order and wait for the product to be delivered and the invoice received.

In any of the cases, approval, rejection or change required the system will send the user a notification.

The process starts when any employee of the organization submits a vacation request.

Once the requirement is registered, the request is received by the immediate supervisor of the employee requesting the vacation.

If the request is rejected, the application is returned to the applicant/employee who can review the rejection reasons.

If the request is approved, a notification is generated to the Human Resources Representative, who must complete the respective management procedures.

The process starts by reviewing the order.

If something is missing, the staff has to inform customer about a potential delay and order the missing products.

After waiting an average of x days for the delivery, the staff should check again if all products are on store.

The courier company informs the client about a potential delay until there are enough human resources to process the delivery.

Response:
select vendor -> F(generate purchase order) AND select vendor -> F(wait for product to be delivered) AND select vendor -> F(receive invoice)

approval, rejection, change -> F(send notification)

START -> submit vacation request

submit vacation request -> F(register requirement)

reject request -> F(return application)

approve request -> F(generate notification)

START -> review order

review order -> F(inform about delay) AND review order -> F(order missing products)

wait for x days -> F(check again)

process delivery -> F(inform)

Prompt:
Now the final 14 sentences:

Once the driver delivers the package, she should update the system so all stakeholders are notified.

Whenever the sales department receives an order, a new process instance is created.

If the storehouse has successfully reserved or back-ordered every item of the part list and the preparation activity has finished, the engineering department assembles the bicycle.

Afterwards, the sales department ships the bicycle to the customer and finishes the process instance.

If the requested amount is lower or equal to 1M$, the company assess the risk of the credit.

After the approval request, the credit could be accepted or rejected

If the requested amount is greater than 1M$, an approval must be requested.

If the requested amount is lower or equal to 1M$, the company assess the risk of the credit.

As soon as an employee files an expense report, the process is started.

Once the expense report is received, a check is performed to make sure the employee has an account.

If this check does not take place within 28 days, the process is stopped and the employee is sent a submit email.

First, the supervisor approves within 7 days, this will result in the deposit of the reimbursement directly to the employee's bank account.

Second, the supervisor rejects within 7 days, as a result the employee receives a notification through email that the request has been rejected.

Third, if it takes the supervisor more than 7 days to respond, an email is sent to the employee informing him/her that the approval is in progress and the part of the processes after a supervisor is asked for approval is repeated.

Response:

deliver package -> F(update system)

Receive order -> F(START)

reserve part -> F(assemble bicycle)

ship bicycle to the customer -> F(END)

Request approval -> F(Accept credit) OR Request approval -> F(reject request)

Request approval -> F(Request approval)

Assess credit risk -> F(Assess credit risk)

File expense -> F(START)

Receive expense report -> F(Perform check)

NOT(check take place) U stop process AND NOT(check take place) U send email

supervisor approves -> F(deposit of reimbursement)

supervisor rejects -> F(receives a notification)

NOT(respond) U send email AND NOT(respond) U supervisor approves OR NOT(respond) U supervisor rejects

Prompt:
you forgot this one:

If the requested amount is lower or equal to 1M$, the company assess the risk of the credit.

Response:
Apologies for that. Here is the LTL constraint for the sentence:

Assess credit risk -> F(START)